

Analysis: Electricity

Test Set 1

For this test set, we can simulate providing electricity to each junction, one junction at a time. For each of these N simulations, we compute how many junctions will end up receiving electricity after the initial junction is powered, and our answer is the maximum of these N simulation results. Each simulation can be accomplished with [depth-first search \(DFS\)](#); we start our DFS at the junction we are directly providing power to. In each step of our DFS, we iterate over the current junction's neighbors, and recurse into the neighbor if its capacity is strictly less than the current junction's capacity. Because the capacity must be strictly decreasing, we are guaranteed to never go in a cycle or visit a junction more than once. The DFS time complexity is therefore bounded by the total number of edges, making it take $O(N)$ time; with N simulations, this gives us an $O(N^2)$ time solution, which is sufficient for this test set.

Test Set 2

We can improve our DFS-based solution from test set 1 to be fast enough for this test set by removing the redundancy among the N DFS invocations and consolidating the computation into an $O(N)$ solution. To give an example of the computational redundancy from our test set 1 solution that we aim to remove, suppose v_1 and v_2 are directly connected junctions, with v_1 having the greater capacity. Simulating providing electricity to v_1 will also lead to simulating providing electricity to v_2 due to the edge between them, but we will also simulate directly providing electricity to v_2 , duplicating our work. We can remove this duplication by treating this as a [dynamic programming](#) problem and [memoizing](#) our work.

Our solution works similarly to before, where for each junction v we perform a DFS to compute how many junctions would be powered if we provided electricity to v . However, we memoize this DFS; when the DFS is called on a junction v , we first check if v is in our memoization table and return the result if it's present. Otherwise, we recurse and perform DFS as usual, but store the result in the memoization table before returning the result. This memoization implies that among our N DFS calls, an edge will never be traversed twice, because after the first traversal, the result will be memoized, making further traversals unnecessary. This implies our N DFS calls will take in total $O(N)$ time, making this approach sufficiently fast for the large test set.

Analysis: Level Design

We have been given a permutation $\mathbf{P}$ of length $\mathbf{N}$ and are asked to find minimum swaps required to form at least one permutation cycle of length K , for all $1 \leq K \leq \mathbf{N}$. Here are a few steps required to solve this problem.

- Step 1: Find the existing permutation cycles.

Considering [permutation cycle](#) as a graph, perform DFS from each unvisited node and keep traversing till we get a visited node. Maintain the cycle length for each DFS. At the end of this we have an array representing cycle lengths of the permutation cycles. Alternative approach is to use [DSU](#) to find all cycles and respective lengths. Let us name the cycle size array as `cycle_sizes`.

- Step 2: Analyze the effect of a swap in the permutation.

Two types of swaps are possible:

- Type 1: Swap within the same cycle, the cycle will be broken into two smaller cycles.
- Type 2: Swap two elements from different cycles, both the cycles will end up merging. The length of the new cycle will be the sum of lengths of two cycles merged.

- Step 3: Optimal merging for each cycle length.

Now we know the existing cycle lengths and the effect of a swap, we need to find the optimal strategy to form a cycle of length K in the minimum number of swaps.

Now the problem essentially boils down to optimally selecting the cycles to merge/break to form a cycle of length K . There are two potential strategies:

1) We can sort the cycle lengths in descending order by length and then merge the cycles starting from the maximum one till we get a final cycle of length $> K$ and then use one more swap to turn that into exactly K . The number of steps required here is an upper bound on the optimal number of swaps and the lower bound will be at least one less than the upper bound (forming cycle of exact length K). Time complexity of greedy approach is $O(\mathbf{N})$ since sorting the permutation cycles itself will take $O(\mathbf{N})$ using [Counting sort](#).

2) We can apply [dynamic programming](#) to optimally find minimum number of cycles required to form a cycle of size exactly K .

The optimal strategy is to choose the minimum of the two strategies.

Test Set 1

For each of the cycles we do have the option either to include it in the optimal set of cycles to be merged or leave it. This problem is now very similar to the [knapsack problem](#).

Let $dp(i, j)$ be the minimum number of swaps required to form a cycle of length j using cycles from indices $(1, i)$.

$$dp(i, j) = \min(dp(i-1, j), 1 + dp(i-1, j - \text{cycle_sizes}(i)))$$

$$S = \text{cycle_sizes.size}()$$

We need to populate the dp table with the above mentioned recurrence relation. $dp(S, K)$ is the answer from the knapsack approach and $\min(1 + dp(S, t))$ for all $K + 1 \leq t \leq \mathbf{N}$ is the answer from greedy approach. Final answer will be minimum of both the values.

Time complexity will be $O(\mathbf{N}^2)$ to populate the dp table.

Test Set 2

Let X be the number of length wise distinct cycles, minimum number of nodes required to form all the X cycles is $\sum_{i=1}^X i$. As per the constraints, $\sum_{i=1}^X i \leq \mathbf{N}$.

Hence there will be $O(\sqrt{\mathbf{N}})$ number of length wise distinct cycles. Using this fact we can speed up our knapsack-based solution from test set 1.

Let us name the distinct cycle size array as `distinct_cycle_sizes` and corresponding occurrences as `cnt`. Let $dp(i, j)$ be the minimum number of swaps required to form a cycle of length j using cycles from indices $(1, i)$ where

$$1 \leq i \leq \text{distinct_cycle_sizes.size}()$$

Hence, in order to calculate $dp(i, j)$ (where i is the index of the current value x being processed and j is the cost), we only need these states $dp(i-1, j-x), dp(i-1, j-2 \times x), \dots, dp(i-1, j - \text{cnt}(i) \times x)$. For each distinct cycle size, store its number of occurrences and then the idea to process v (occurrences of each cycle) items of value x is to treat the problem as a [sliding window minimum](#) problem.

$$\text{Hence, the recurrence becomes } dp(i, j) = \min(v + dp(i-1, j - v \times \text{distinct_cycle_sizes}[i]), v \in [0, \text{cnt}(i)])$$

Then, if we consider the values $j \bmod k$, you will notice that for a fixed remainder it just becomes a range min query on a 'sliding window' interval (namely, the left bound of the interval may only move to the right each query), which can be computed in amortized constant time using a monotonic deque.

So, now for each $i \in [1, \mathbf{N}]$, the optimal answer = $\min(dp(\text{items_count}, i), dp(\text{items_count}, j) + 1)$ where $j > i$.

Complexity will be $O(N\sqrt{N})$ to populate the dp table.

Sample Code (C++)

```
const int inf = 1e9;
void knapsack(vector<int> distinct_cycle_sizes, vector<vector<int>> dp, vector<int> cnt, int n) {
    dp[0][0] = 0;
    for(int i = 1; i <= (int)distinct_cycle_sizes.size(); ++i) {
        int cs = distinct_cycle_sizes[i - 1];
        dp[i][0] = 0;
        // Using standard sliding window approach.
        for(int j = 0; j < cs; ++j) {
            deque<int> dq;
            for(int l = j; l <= n; l += cs) {
                dp[i][l] = min(dp[i][l], dp[i - 1][l]);
                int opt = inf, x = -1;
                if(l and !dq.empty()) {
                    opt = dp[i - 1][dq.front()] + (l - dq.front()) / cs;
                    x = dq.front();
                } else if(!l) {
                    opt = 0;
                }
                dp[i][l] = min(dp[i][l], opt);
                while(!dq.empty() and dq.front() <= l - cnt[cs] * cs) {
                    dq.pop_front();
                }
                while(!dq.empty() and dp[i - 1][l] <= dp[i - 1][dq.back()]) {
                    dq.pop_back();
                }
                dq.push_back(l);
            }
        }
    }
    vector<int> fans(n + 1);
    int best_right = inf, items_count = distinct_cycle_sizes.size();
    for(int i = n; i >= 1; --i) {
        // best_right + 1 accounts for greedy.
        fans[i] = min(dp[items_count][i], best_right + 1);
        best_right = min(best_right, fans[i]);
    }
    for(int i = 1; i <= n; ++i) {
        cout << fans[i] - 1 << " ";
    }
    return;
}
```

Analysis: Magical Well Of Lilies

Test Set 1

For the small test set, a simple backtracking algorithm is sufficiently fast. In each recursive call, we need to keep track of the number of lilies remaining in the well, the current state of well's memory, and the number of coins we have tossed so far. In each recursive step, we have at most three choices:

- Toss one coin for another lily;
- Toss two coins for as many lilies as last noted by the well;
- Toss six coins ($4 + 2$) for the well to remember the current state of the basket and then toss as many lilies.

Of course, we would consider an operation only if the well has that many lilies, for it would be a waste of coins otherwise. If the number of lilies left in the well becomes 0 at any time of the recursion, we update the minimum number of coins, if necessary, and return immediately from the recursion.

Since we have at most three choices at each recursion level, the time complexity of this exhaustive search might seem like a prohibitive $O(3^L)$, however, the algorithm is much faster in practice for two reasons. First, not all three choices are available at each recursion level. And second, the maximal recursion depth of L is possible only if we pick up the lilies one by one. In other words, the average recursion depth is much smaller than L . For example, the algorithm needs just 5060 recursive calls for $L = 20$.

Test Set 2

For the large test set, the exhaustive search is obviously too slow as the number of recursive calls for $L = 50$ is already 7,821,316,841. Fortunately, dynamic programming comes to rescue here.

But first, let us make one useful observation. Namely, we can always rearrange some operations such that a two coin operation never follows a one coin operation. If it does, we can always swap the two operations without affecting the net effect of the whole sequence of operations.

Now let us turn to our dynamic programming recurrence. Let $DP[i]$ be the minimum number of coins needed to fetch i lilies from the well. We have $DP[0] = 0$ and $DP[1] = 1$. For $i > 1$, there are two possible ways of arriving at i lilies — we either collect $i - 1$ lilies first and then use a one coin operation to get another lily, or the last operation must have been a two coin operation. The latter means that we had asked the well to remember the state of our basket at k lilies (where k is a divisor of i) and then applied $\frac{i}{k} - 1$ two coin operations. Formally,

$$DP[i] = \min(DP[i - 1] + 1, \min_{k|i} \{DP[k] + 4 + 2(\frac{i}{k} - 1)\}).$$

In practice, instead of calculating each $DP[i]$ directly using the formula above and iterating over the divisors of i , we inverse the process as follows:

At the beginning we can say that all values of DP except for 0 and 1 are equal to infinity. Now let us iterate through all i from 2 to N . One option to use one coin and get from $i - 1$ lilies to i lilies, so we can say that $DP[i] = \min(DP[i], DP[i - 1] + 1)$. Now when we already have i

lilies, we can also use four coins to remember the state of our basket and then later get $j \cdot i$ lilies from the well. So we should update DP for all multiples of i :

$DP[j \cdot i] = \min(DP[j \cdot i], DP[i] + 4 + 2 \cdot (j - 1))$ for $i < j \cdot i \leq \mathbf{N}$.

As for the time complexity of the algorithm, for each index k , there are $O(\frac{\mathbf{L}}{k})$ multiples to be updated, which leads to $O(\mathbf{L}(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{\mathbf{L}}))$ updates. The factor $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{\mathbf{L}}$ is the $\mathbf{L}$ -th [harmonic number](#), which is $O(\log \mathbf{L})$, therefore, the overall time complexity of the algorithm is $O(\mathbf{L} \log \mathbf{L})$.

As a final note, we can precompute the DP table in advance, and then answer each test case in constant time.

Analysis: Running in Circles

Test Set 1

Since Ada only runs in the clockwise direction, we can ignore the direction input altogether as Ada will always cross the starting line in the same direction she had last crossed it. Now, we can calculate the total distance run by Ada by calculating the sum:

$$dist = (D_1 + D_2 + \dots + D_N)$$

This sum can be calculated while reading the input line-by-line and adding to $dist$, so we do not need to store the value of each D_i . Then, we can calculate $\lfloor \frac{dist}{L} \rfloor$, and obtain the number of laps the machine has counted.

$$laps = \lfloor \frac{dist}{L} \rfloor$$

Time and Space Complexity: This solution will run in $O(N)$ time and take up $O(1)$ extra space.

Test Set 2

Going through the input line-by-line, we calculate how much Ada has to run to be able to reach the starting line again (that is, the remaining distance to starting line, R) from the Ada's current position pos ($pos = 0$ initially) before a run and in the current direction C_i . We also keep track of the previous direction $prevdir$ she was running in when she last crossed the starting line.

R can be calculated as follows:

- If C_i is clockwise, $R = (L - pos) \bmod L$
- If C_i is anticlockwise, $R = pos$

During each run (that is, for each line of input):

- If $D_i < R$, then we will just update her position pos , as she can not reach the starting line.
 - If C_i is clockwise, $pos = pos + D_i$
 - If C_i is anticlockwise, $pos = pos - D_i$

After this update, we continue to the next run without updating $prevdir$ as Ada has not crossed the line this time.

- On the other hand if $D_i \geq R$, and if $R > 0$, we increase the number of laps by 1 if C_i is the same as $prevdir$. However, if $R = 0$ (that is, Ada is exactly at the starting line), we do not perform this increment since Ada had already reached the starting line in the previous run, where it was counted as a lap by the machine (similarly for $R < 0$). Thus, we do not treat starting a run at the line as "crossing" the line, rather, we count a lap when Ada finishes at the line. With this logic, the case where Ada changes her direction on the line itself will also not be counted as a lap.

We then increase the number of laps by the value $\lfloor \frac{D_i - R}{L} \rfloor$ to take care of cases where multiple laps are made in a single run (this will also handle the above case, where Ada just reaches the starting line after a run). As these laps are counted from the starting line, $prevdir$ does not matter in this calculation. Then, we update Ada's current position:

- If C_i is clockwise, $pos = (D_i - R) \bmod L$
- If C_i is anticlockwise, $pos = L - ((D_i - R) \bmod L)$

After the above process, we update *prevdir* to C_i and continue.

Finally, return the number of laps counted.

Time and Space Complexity: This solution also runs in $O(N)$ time, and takes up $O(1)$ extra space.